

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 6.2a: Why ML for HA Models and Euler+ML

Zhigang Feng

Spring 2026

Topic 6.2a Agenda

1. **Why ML? Motivating Example**
2. **Euler Equation + ML**

*Arc: T1 Aiyagari + computation → **T2a Why ML / Euler** → T2b Global DNN → T3 KS / DeepHAM → T4 DEQN / OLG → T5 MFG → Lab06*

Why ML? Motivating Example

Why approximate the value function with a neural network?

- Grid-based methods scale exponentially: N_a^d points for d continuous dimensions.
- Even modest models (income $\times$ wealth $\times$ housing $\times$ aggregate) become intractable.
- **Neural networks:**
 - Universal approximators with smooth basis functions.
 - Number of parameters scales (essentially) linearly in input dimension.
 - Compatible with stochastic gradient training and GPU hardware.
- Strategy: parameterise V and the policy σ by neural networks Γ_{γ_v} and Γ_{γ_σ} , then update $(\gamma_v, \gamma_\sigma)$ by training.
- We illustrate first on a simple deterministic optimal growth model, then scale up.

Optimal growth as a toy: NN approximations of V and σ

The deterministic Bellman equation:

$$V(k) = \max_{\sigma} \{ u(k, \sigma(k)) + \beta V(f(k, \sigma(k))) \}.$$

Parameterise:

$$V(k) \approx \Gamma_{\gamma_v}(k), \quad \sigma(k) \approx \Gamma_{\gamma_{\sigma}}(k).$$

Algorithm sketch:

- Sample states $k \sim d(k)$.
- Compute target values $\hat{V}(k)$ from the right-hand side using current networks.
- Update γ_v to minimise $(\Gamma_{\gamma_v}(k) - \hat{V}(k))^2$.
- Update γ_{σ} by gradient ascent on the per-state objective.
- Iterate.

One-step value update ($T_{\text{sim}} = 1$)

Given current policy $\sigma^{(n-1)}$ and value $\Gamma_{\gamma_v}^{(n-1)}$, define the bootstrap target

$$\hat{V}(k) = u(k, \sigma^{(n-1)}(k)) + \beta \Gamma_{\gamma_v}^{(n-1)}(f(k, \sigma^{(n-1)}(k))).$$

Then update by minimising the mean-squared Bellman residual:

$$\gamma_v^{(n)} \in \arg \min_{\gamma_v} \mathbb{E}_{k \sim d} \left[\left(\Gamma_{\gamma_v}(k) - \hat{V}(k) \right)^2 \right].$$

- Estimate the expectation by a Monte Carlo average over a batch $\{k_i\}_{i=1}^B$.
- One *epoch* = several gradient steps on the loss.
- This is essentially *fitted value iteration* (Bertsekas & Tsitsiklis 1996), with a NN function class.

One-step policy update

Given the just-updated value $\Gamma_{\gamma_v}^{(n)}$, improve the policy by maximising the per-state RHS over the parameters of the policy network:

$$\gamma_{\sigma}^{(n)} \in \arg \max_{\gamma_{\sigma}} \mathbb{E}_{k \sim d} \left[u(k, \Gamma_{\gamma_{\sigma}}(k)) + \beta \Gamma_{\gamma_v}^{(n)}(f(k, \Gamma_{\gamma_{\sigma}}(k))) \right].$$

- Gradient with respect to γ_{σ} is computed automatically (backpropagation).
- One step of Adam (or any first-order optimiser) on a Monte Carlo batch.
- Alternating these updates is structurally identical to **actor–critic** reinforcement learning.

Choosing the sampling distribution $d(k)$

Question: on which states k should we evaluate the loss?

- *Uniform* on the bounded grid: simple but wasteful (most weight on irrelevant states).
- *Stationary distribution under the current policy*: focuses learning where the model actually spends time. Computed by simulation.
- *Importance sampling*: re-weight to keep the gradient unbiased while sampling from a convenient proposal $q(k)$:

$$\nabla_{\gamma} \mathbb{E}_d[L(k; \gamma)] = \mathbb{E}_q \left[\frac{d(k)}{q(k)} \nabla_{\gamma} L(k; \gamma) \right].$$

The choice of $d(k)$ has *first-order* implications for solution accuracy: a network trained on the wrong support generalises poorly.

Bias–variance tradeoff for $T_{\text{sim}} = 1$

Pros of one-step bootstrapping:

- Cheap: each evaluation costs a single network forward pass.
- Low variance.

Cons:

- High bias: the target $\hat{V}$ relies on the previous (imperfect) network.
- Errors compound across iterations if the network is far from the true V .

Multi-step rollouts ($T_{\text{sim}} > 1$):

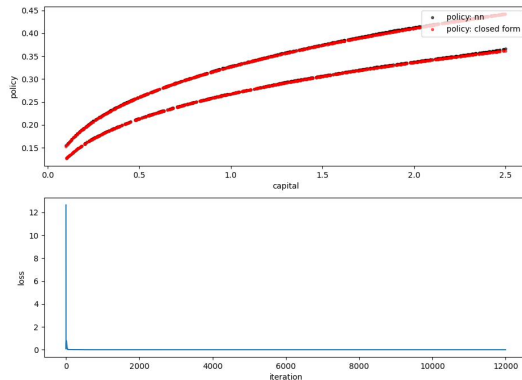
$$\hat{V}(k) = \sum_{t=0}^{T-1} \beta^t u(k_t, g(k_t)) + \beta^T \Gamma_{\gamma_v}(k_T).$$

- Reduces bias (less reliance on the bootstrapped tail).
- Increases variance and per-step cost.
- Choose T to trade off accuracy vs. wall time.

Improvement: stationary-distribution sampling

- Naive uniform sampling over a large bounded grid is wasteful: most points are never visited in equilibrium.
- Better: simulate the economy under the current policy until it converges to its (induced) stationary distribution, then sample k from that distribution.
- Benefits:
 - *Sample efficiency*: training data concentrates where the model lives.
 - *Faster convergence*: gradients informative.
 - *LLN*: large simulation $\Rightarrow$ low-variance estimates of expectations.
- Caveat: the sampling distribution *depends on the current policy*, which itself is being trained — creates a moving target. Standard fix: refresh the simulation every few epochs.

An illustration: optimal growth under stochastic productivity



- Stationary distribution of capital under the trained policy, vs. analytical benchmark.
- NN-trained policy matches the benchmark closely after a few thousand epochs of alternating updates.

What does RL/DL give us beyond grids?

- **Function approximation that scales with data, not with grid size.** A 4-layer MLP with a few thousand parameters can represent value functions in $\mathbb{R}^{20}$ that no Cartesian grid could store.
- **Stochastic gradient training.** Each gradient step uses a small batch; we can train indefinitely on streaming Monte Carlo data.
- **Hardware acceleration.** Forward and backward passes are GPU-friendly matrix multiplications.
- **End-to-end differentiability.** Bellman residuals, Euler residuals, and even simulation steps can be differentiated through.
- These advantages compound when we move from optimal growth $\rightarrow$ Aiyagari $\rightarrow$ Krusell–Smith $\rightarrow$ HANK.

Euler Equation + ML

Euler equation as an alternative optimality condition

For the optimal growth model with $u'(c) > 0$ and the budget constraint $c + k' = f(k)$, the consumer's first-order condition is the **Euler equation**:

$$u'(c(k)) = \beta u'(c(k')) f'(k').$$

- Equivalent to the Bellman equation (under standard regularity); equivalence to the optimum also requires the transversality condition.
- Does not require representing V explicitly: we can train only the policy network.
- For the stochastic case with shock z :

$$u'(c(k, z)) = \beta \mathbb{E}[u'(c(k', z')) (1 + r(k', z')) \mid z].$$

From Euler residual to MSE loss

Define the **Euler residual** at state k as

$$\varepsilon(k; \gamma_\sigma) = 1 - \frac{\beta \mathbb{E}[u'(c'(k; \gamma_\sigma)) (1 + r') \mid k]}{u'(c(k; \gamma_\sigma))}.$$

The optimal policy makes $\varepsilon(k) \equiv 0$. Cast as supervised learning:

$$\gamma_\sigma^* \in \arg \min_{\gamma_\sigma} \mathbb{E}_{k \sim d} [\varepsilon(k; \gamma_\sigma)^2].$$

- Loss is *differentiable* with respect to γ_σ via autograd.
- Conditional expectation approximated by Monte Carlo over future shocks.
- No explicit value function required.

Minimising the Euler-residual loss

Algorithm:

1. Initialise γ_σ randomly (e.g., Xavier/Glorot).
2. Sample a batch of states $\{k_i\}_{i=1}^B$ from $d(k)$.
3. For each k_i , compute $c_i = c(k_i; \gamma_\sigma)$ and $k'_i = f(k_i) - c_i$.
4. Sample shocks z' and compute the Monte Carlo expectation in ε .
5. Compute the squared residual loss and its gradient wrt γ_σ .
6. Update γ_σ using Adam.
7. Repeat until convergence ($\sum_i \varepsilon_i^2 < \text{tol}$).

Why this is attractive: *first-order necessary conditions* are precisely what we want the policy to satisfy. The neural network is just a flexible function class.

Code 1/5: ValueNet architecture (PyTorch)

```
import torch
import torch.nn as nn

class ValueNet(nn.Module):
    """Two-hidden-layer MLP V_gamma(k)."""
    def __init__(self, n_hidden=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(1, n_hidden),
            nn.ReLU(),
            nn.Linear(n_hidden, n_hidden),
            nn.ReLU(),
            nn.Linear(n_hidden, 1),
        )

    def forward(self, k):
        return self.net(k)
```

Code 2/5: value update step (Bellman MSE)

```
def value_update_step(value_net, policy_net, k_batch,
                      beta, opt_v):
    """One gradient step on the Bellman MSE loss."""
    opt_v.zero_grad()
    c_now = policy_net(k_batch)
    k_next = production(k_batch) - c_now
    v_next = value_net(k_next).detach()
    target = utility(c_now) + beta * v_next
    v_now = value_net(k_batch)
    loss = ((v_now - target) ** 2).mean()
    loss.backward()
    opt_v.step()
    return loss.item()
```

Note the `.detach()` on `v_next`: we treat the bootstrap target as a constant.

Code 3/5: policy improvement (gradient ascent)

```
def policy_step(value_net, policy_net, k_batch,
                beta, opt_g):
    """Maximise  $u(c) + \beta V(k')$  with respect to policy params."""
    opt_g.zero_grad()
    c = policy_net(k_batch)
    k_next = production(k_batch) - c
    objective = (utility(c) + beta * value_net(k_next)).mean()
    loss = -objective # gradient *ascent*
    loss.backward()
    opt_g.step()
    return objective.item()
```

The budget constraint $c + k' = f(k)$ is encoded inside the function so the policy net only outputs c .

Code 4/5: training loop

```
def train(value_net, policy_net, n_epochs, batch_size,
          beta, lr_v=1e-3, lr_g=1e-3):
    opt_v = torch.optim.Adam(value_net.parameters(), lr=lr_v)
    opt_g = torch.optim.Adam(policy_net.parameters(), lr=lr_g)

    history = {"v_loss": [], "obj": []}
    for epoch in range(n_epochs):
        k_batch = sample_states(batch_size)          # from d(k)
        v_loss   = value_update_step(value_net, policy_net,
                                     k_batch, beta, opt_v)
        obj      = policy_step(value_net, policy_net,
                               k_batch, beta, opt_g)
        history["v_loss"].append(v_loss)
        history["obj"].append(obj)
    return history
```

Code 5/5: visualisation

```
import matplotlib.pyplot as plt

def plot_results(value_net, policy_net):
    k = torch.linspace(0.1, 5.0, 100).reshape(-1, 1)
    with torch.no_grad():
        v = value_net(k).squeeze().numpy()
        c = policy_net(k).squeeze().numpy()

    fig, ax = plt.subplots(1, 2, figsize=(10, 4))
    ax[0].plot(k.numpy(), v); ax[0].set_title("V(k)")
    ax[1].plot(k.numpy(), c); ax[1].set_title("c(k)")
    plt.show()
```

A few thousand epochs of alternating value/policy updates produce policies that match the analytical benchmark to within numerical tolerance.

Actor–critic perspective: this is RL

Recall: Lec04 T2 previewed actor–critic networks for DSGE solvers; Lec05 T3 formalised the family (A2C / PPO / DDPG / SAC). The two-network setup here — ValueNet (critic) trained on Bellman MSE; policy net (actor) trained on the gradient of $\hat{V}$ — is the simplest concrete instance in the course.

- The two-network alternating update is exactly the structure of **actor–critic** reinforcement learning:

$$\text{Critic: } \Gamma_{\gamma_v} \leftarrow \arg \min \|\Gamma_{\gamma_v} - \hat{V}\|^2,$$

$$\text{Actor: } \Gamma_{\gamma_\sigma} \leftarrow \arg \max \mathbb{E}[u + \beta V].$$

- Difference vs. classical RL: in macro the environment (the model) is *known*, so we can compute targets analytically rather than by Monte Carlo over an unknown $P(s'|s, a)$.
- This makes deep-learning macro *model-based RL with a known model* — often called “deep dynamic programming”.

Next (T2b): global DNN solution when the full cross-sectional distribution becomes part of the state